

Physical Database Design

⇒ The physical methodology is the third and final phase of the database methodology.

⇒ Here, the designer must decide how to translate the logical database design (entities, attributes, relationship, and constraints) into a physical database design, that can ultimately be implemented using the target DBMS.

~~Def~~ ⇒ "Physical database design is the process of making a description of the execution of the database on secondary storage, which describes the base relations, file organizations as well as indexes used to gain efficient access to the data."

and any associated integrity constraints and security measures.

<u>Logical database:</u>	<u>Physical database</u>
Concerned with "What".	Concerned with "How".

=> Physical database design requires diverse skills that are often found in different people.

=> Physical database designer should have knowledge about how computer system hosts the DBMS and how it should operates and must be fully conscious of the working of the Target DBMS.

Steps required for implementing physical methodology:

Physical database design methodology
are as follows:

⇒ Transform the logical data model
for target DBMS

- Design base relations
- Design representation of derived data
- Design general constraints

⇒ Design file organizations and
indexes

- Analyze the transactions
- Choose file organizations
- Choose indexes
- Estimate disk space requirements
- Design user view
- Design security mechanism
- Consider the introduction of
controlled redundancy.
- Monitor and tune the
operational system.

Designing fields:

A field is the smallest unit of application data recognized by system software, such as a programming language or DBMS.

A field corresponds to a simple attribute in the logical data model.

Choosing data types:

A data type is a detailed coding scheme recognized by system software, such as a DBMS, for representing organizational data.

Selecting a datatype involves four objectives that will have different relative levels of importance for different applications:

- 1) Represent all possible values.
- 2) Improve data integrity.
- 3) Support all data manipulation.
- 4) Minimize Storage Space.

o Coding & Compression techniques:

- ⇒ There are some attributes which have some sparse set of values.
- ⇒ These values are very difficult to express when they are represented in any data type, for this purpose some codes are used.
- ⇒ A field with limited number of possible values can be translated into a code that requires less space.
- ⇒ These codes are defined by the db administrator or the programmer consume less

space so they are better for use in situations where we have large numbers of records and wastage of small amount of space in each record can lead to loss of huge amount of data storage space.

Coding techniques are also useful for compression of data values appearing in the data, by replacing those data values with the smaller sized codes that will result in reduction of storage.

ST ID	Sname	Hobby
S1020	Sohail das	Reading
S1038	Shoaib Ali	Gardening
S1015	Tahira Ejaz	Reading
S1015	Tahira Ejaz	Movie
S1018	Ash Zia	Reading

Coding example:

Stid	SName	Hobby
S1020	Sohail das	R
S1038	Shoaib Ali	G
S1015	Tahira Ejaz	R
S1015	Tahira ejaz	M
S1018	Ash zia	R

by Table:-

Code	Hobby
R	Reading
G	Gardening
M	Movies

Controlling data integrity:

For many DBMS, data integrity controls can be built into the physical structure of the fields and controls enforced by the DBMS on those fields.

The data type also enforces some type of data integrity.

Control because it limits the type of data and the length of a field value.

Some typical integrity controls that DBMS may support:

Default values

A default value is the value of a field will assume unless a user enters an explicit value for an instance of that field.

Assigning a default value to a field can reduce the data entry time because entry of a value can be skipped.

Also help to reduce data entry errors for the most common value.

Range Controls

Limits the set of permissible values a field may assume.

- o Range may be a numeric lower - to upper bound or set of specific values.
- o Range controls must be used with caution because the limits of the range may change over time.
- o for example, A combination of range controls and cooling led to the year 2000 problem that many organizations faced, in which a field of years represented by only the numbers 00 to 99.
- o So, range control must be implemented through a dbms because range control in many applications may be inconsistently enforced.
- o it is also more difficult to find and change them in application rather than a dbms.

Null value control:

A null value is an empty value.

Each pk must have an integrity control.

Any other field may also have a null value control on it if that is the policy of an organization.

Referential integrity

How to handle missing data in physical data-base design?

Two options for handling or preventing missing data.

- o Using a default value
- o Not permitting null value

Missing data are inevitable.

According to Baber & Hoffer 1984, the following are some methods for

handling

- 1) Substitute the missing
 - 2) Constant missing val
 - 3) In prog data unless significant
- ⇒ Trigger to perform the

handling missing data:

Substitute an estimate of the missing value (Using a formula).

Construct a report listing missing values.

In programs, ignore missing data unless the value is significant.

Triggers can be used to perform these operations.

Denormalization

- ⇒ Denormalization is a database optimization technique in which we add redundant data to one or more ^{normalized} tables.
- ⇒ This can help us avoid costly joins in a relational database.
- ⇒ it is applied after normalization.
- ⇒ it is basically, the process of taking a normalized schema and making it non-normalized.
- ⇒ Designers use it to tune the performance of systems to support time-critical operations.

Pros:

- Retrieving data is faster since we do fewer joins.
- Queries to retrieve can be simpler.

Cons

- o Updates and inserts are more expensive
- o Data may be inconsistent
- o Data redundancy requires more storage

Difference between normalization & denormalization

Normalization	Denormalization
<u>Definitions:</u>	
o The process of decomposing relations with anomalies to produce, smaller well-structured relations.	o The process of transforming normalized relations into non-normalized physical record Specification.
<u>Implementation:</u>	
o Used to remove redundant data from the DB.	o Process of adding redundant data to a database

Purpose:

To reduce the
data redundancy
& inconsistency.

o To achieve
consistency of
data through
introducing rules.

Data integrity:

Maintained

o may not
maintained.

Number of tables

Increases

o decreases

Disk space:

Optimized usage

o - wastage

When to use?

where a number
of insert delete,
update operations
are performed
and joins of
those tables &
joins of those
tables are not
expensive.

o where joins
are expensive
and frequently
executed on
the tables.

example:

products categories

- id
- name
- category id

- id
- name

orders

- id
- details
- amount
- pid
- cid

customers

- id
- name
- email

customer orders

- id
- Pname
- Pcode
- Cname
- customer name
- customer email
- orders id
- O details
- O amount

Partitioning:

- Simply a way of choosing the separate physical storage locations for a single database object
- Before partitioning was implemented : by database vendors, this was done by creating separate tables for current and historical data using views to combine the partitioned data.
- Enhance the performance, manageability & availability of wide variety of applications & help reduce the cost of ownership.

- Allows table index etc to be subdivided into smaller pieces
- Partitioning key is composed of one or more columns that determine the partitions into which some data will be stored.

- o Records used together are grouped together for efficiency
- o Each partition can be optimized for performance
- o Security, security
- o Partitions stored on different disks, contention
- o Take advantage of parallel processing

Cons

- o Inconsistent access speed
- o Complexity
- o Extra space & update time

Horizontal & Vertical Partitioning:

<u>Horizontal</u>	<u>Vertical</u>
o Distributing the rows of a table into several separate files, based on common values	o Distribution of the columns of a logical relation into separate physical tables

o useful when there is more critical & sensitive data.

when to use?

where different users need to access of different rows.
PK repeated but half values.

o where different users need to access to different columns.
o PK must be repeated with each column.

Example

V1		
cid	Fname	Lname

V2	
cid	City

cid	Fname	Lname	city
1	Alice	Anderson	Austin
2	Bob	Bost	Boston

cid	Fname	Lname	city
3	Carrie	Conway	Chicago
4	David	Doe	Denver

Physical database file & tablespace

Physical file	tablespace
A physical file is a named portion of secondary	=> A tablespace is a named

memory (such as magnetic tape, hard disk or solid state disk) allocated for the purpose of storing physical records.

Entents:

=> Some computer operating system allow a physical file to be split into separate pieces, sometimes called entent.

=> Physical files contain the actual data that is stored on the system, and a description of how data is to be presented to or reserved from a program.

Logical storage unit in which data from one or more database tables, views or other database objects may be stored.

o A table-space consists of one or more physical system files.

=> They are used to organize data in a database. Logical storage grouping table data.

Benefits of tablespace

- Recoverability
- Automatic Storage management
- memory utilization.

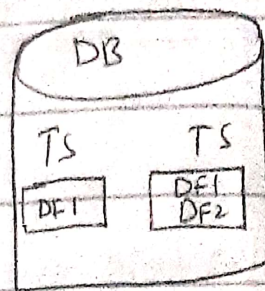
⇒ Each database must contain at least three tablespaces as given here:

- Catalog tablespace
- User tablespace
- Temporary tablespace

⇒ A tablespace is an object in the database that maps a single file to one or more tables.

Syntax:

```
Create Tablespace TName  
[ Add datafile 'FileName'];  
Create Table TName  
( Colname col definition, ... )  
[ Tablespace TablespaceName]
```

Shard:

A shard is a subset of a table in a distributed database.

Partition types:

- 1) Range
- 2) List
- 3) Hash or key partition

Partitioning criteria

Range partitioning:

in which each partition is defined by a range of values (lower & upper key value limits) for one or more columns of the normalized table.

maps data to partition based on range of partition keys values established for each partition

⇒ This partitioning allows us to partition the rows of a table based on column values that fall within specified range.

⇒ The given range always in Contiguous form but should not overlap each other.

⇒ Also uses the VALUES less than Operator to define the ranges or MAX value for max.

⇒ It is normally used with dates.

⇒ A partition key may be generated by the database designer to create more balanced distributions of rows.

List partitioning:

in which the partitions are defined based on predefined lists of values of the partitioning key.

- Enables to control how rows map to partitions.
- By providing discrete values for the partition key in description of each partition.

for example
A table is partitioned based on the value of the Column State, one partition might includes rows that have the value "CT", "ME", "RI", "NH" or "VT" etc.

⇒ Some points to consider when using list partitioning include:

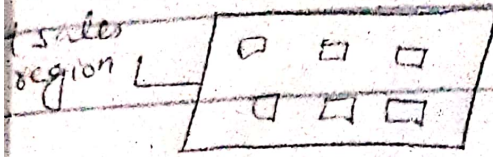
1) The partitioning key can only consist of a single table column.

2) It is easy to group and organize unindexed and related data sets in different partitions.

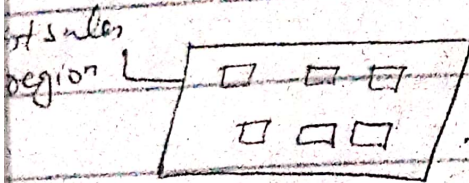
3) We can partition data based on specific columns such as region.
examples:

o Region: india, ID number: 3, 4

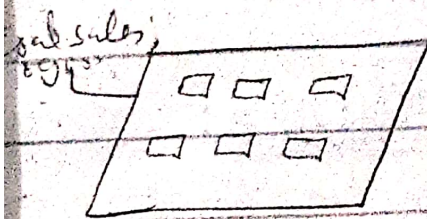
List partitioning



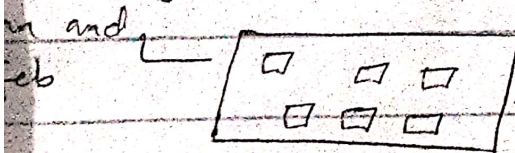
Partition by List (date)



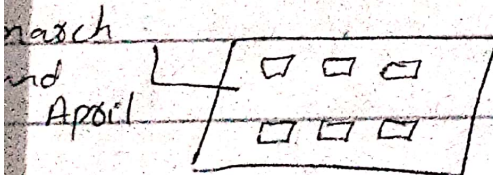
Partition with values
(C1, C2, C3...Cn)



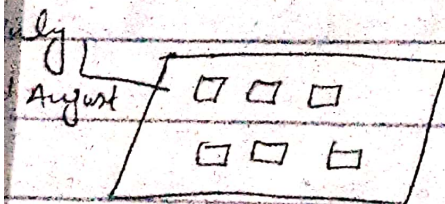
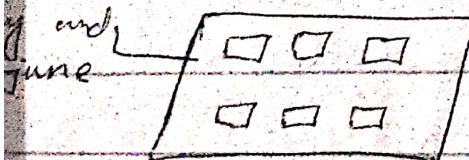
Range partitioning



Partition by Range
(date)

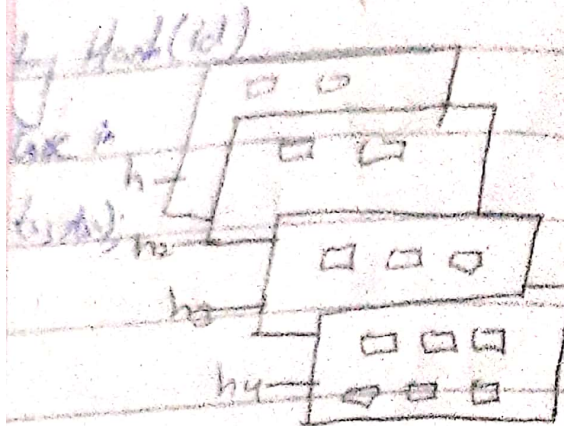


Partition Sale-Jan
values less than
(TO-date ('02/08/2015



mm/dd/yyyy);

Hash partitioning



Hash partitioning:

Create Partitions in Round-Robin manner

In hash partitioning, rows are divided into different partitions based on hashing algorithm.

This is different from range partitioning, which groups database rows based on continuous indexes.

Hash partitioning can be used in a few different ways.

We can use a client identifier, IP address or application ID as an input

to a hash function to generate a hash value.

This value determines which database partition to use for the request.

for example,

if we have 4 database partitions and each request contains an application ID, we could perform a modulo operation on the application ID with 4 and take the remainder to determine the partition to use.

it is easy to use alternative to range partitioning.

Drawbacks

o Expensive to dynamically add or remove database servers.